24303 DATABASES – LABS

Challenge 1: Database design and implementation

Goals:

The topics of this challenge are:

1. Requirement analysis and scenario comprehension
2. Conceptual Model (Entity-Relation) diagram
3. Relational Model diagram
4. Database implementation SQL (DDL)

Methodology:

The challenge must be read in order, following the instructions given step by step. You are also provided in AG a LAB0 as an example for how your delivery is expected (syntax, read

Grouping:

The challenge must be developed in teams of **three students** of your choice. Groups can not be changed between challenges and should be registered previously using the method provided (form submit). If you did not fill in the form you were randomly assigned to a team.

LAB Database Server access:

You will be provided (between 1st and 2nd LAB class with a database server connection to deploy your database (if you are listed in a LAB Team Group).
Although the database server is not needed on the first two phases of this first challenge, so you don't need to ask for access on your first lab class.
Keep in mind that this server access will be used also in Challenge 2.

For the meanwhile, ensure that you are able to connect from home using UPF's VPN connection: https://www.upf.edu/intranet/intern-universitat/connexio-vpn

Doubts:

For any doubts outside LABs hours regarding the challenge, please contact always both LAB professors in your emails and we will come back to you asap:

- guillem.casacuberta@upf.edu
- viktoriia.popova@upf.edu

WELCOME TO THE NBA

The National Basketball Association is well known as the best basketball league in the World and it's getting bigger and bigger year by year. Because of that, they need to store their data in a better way in order to scale up.

They opened a challenge to everyone who feel able to propose a better solution than the one they have right now. The proposal is about designing a new database to be delivered with a Conceptual and a Relational Model as well with the needed SQL code to install it. Will it be you?

As we may know, the game is all about the players in the court, so we need to store the following information related to the hoopers: its ID card, its name and surname, its nationality, the gender, their age, the year they turned PRO, from which university they have been drafted and the amount of NBA championships they have.

The athletes play for a certain franchise at a time (*franchise is the way teams are referred to in the NBA*). They can change their team, but they can only play for a team at a time. Nevertheless, the players can be selected also for their national team, but not all of them will reach it. Regarding the franchises we would like to store the information of their name (from which would be identified), the city they play for, its annual budget and the amount of NBA's championship rings it has.
For the national teams, we would like to have the info of the country they play for and the year of the roster selection.

As we can imagine, a player can be selected for different consecutive years to play for the national team while the national team is filled with many players every year.



Regarding the franchise and the players, we would like to store the information of their shirt number, the date when the player signed in and the year they signed out and also their salary. All this information should be stored for every player and for every team they played for.
For instance, Luka Dončić is been playing many years for Dallas Mavericks wearing the number 77 and with a certain salary. But in the future he may sign for another team, wear another number in the jersey and earn a different amount of salary; such as many other players have done before like Luka's teammate Kyrie Irving. The database design should be able to keep this information.

Each summer before the beginning of every season, there is a draft process where around 58 new rookie players are selected from their university and assigned to a team.
The better the player's perspective is, the better position he has in the draft list. The player in the first draft position is expected to be the best rookie player of the next season.

With the intention of making the NBA league more fair, the less competitive teams of the last season, will take part in the draft lottery and will have the right of the firsts choices over the draft players list in order to get the best players they need. The draft lottery's purpose is to determine the order of choice of those teams.
In the following image we can see the Atlanta Hawks being the first team which gets to choose a player because it got the first position in the draft lottery among all the other less competitive teams of last season.



Some of the NBA players, but not all of them, may have been drafted for a certain team in their university's rookie year. We need to store this information for every player who has been drafted and also the rank (position) on the draft list. We will keep the drafts sorted and identified by its year.

The players who have not been ever been in a draft list is because they have been playing overseas in Europe, or because they come from other minor leagues around the world, or because when they

were playing at the university and it was their time to be selected, they were not considered for any team as an interesting player for the NBA. Take into account that the draft players list is only around 60 players each year.

We also have the information of head coaches and assistant coaches. For both of them we need to store the information of its ID card, its name and surname, its nationality, the gender and their age. We need to store the percentile of victories (%) and the salary for the head coaches and the speciality for the assistant coaches. Moreover, some of the assistant coaches will be the boss of other assistant coaches.

A head coach can train a franchise and also a national team while all the teams will have only a head coach. The assistant coaches work always for the same franchise while a franchise can have several of them. For the national teams, we do not have the information of who their assistant coaches are; so, we won't store that information.

Each franchise is part of either the western conference or the eastern conference. We would like to store the name of the conference which cannot be repeated and its geographical location.

Also, each franchise plays in the regular season every year. For the regular seasons we want to store the year (e.g., 2026-2027) with which we will identify them, and the dates of start and end. We want to know whether a team wins the championship ring or not for every year.



All this happens in the stadiums, so we need to store the information of the arena's name (fi: Staples Center), the city (fi: Los Angeles) and the capacity (fi: 20K). Every arena is composed of some zones and each zone has seats. For the zone we know its code and if it's VIP or not. For the seats we only know a number and its colour. We have to take into account that the seat number can be repeated among the zones, so it wouldn't be enough to identify a seat by itself.



Apart from players and coaches, there are many other people working in the league: cleaning staff, ticket sales staff, security guards, the mascot, bartender staff, etc. We would like to store the information regarding those groups of workers also.



For all of them we need to store the information of their national id, birthdate, city of residence, bank account number, the type of contract which can be full time or part time and for which franchise they are working nowadays.

Also each worker groups will have their own specifications: for the cleaning staff we need to know how fast they can run in miles per hour in order to be able to clean the floor while the game is on, for the ticket sales group we need to know that they have no gambling record, for the security

guards we would like to know whether they have a gun licence, for the mascot, we need to know which custom animal they prefer to wear and for the bartenders we need to know if they have any record of being caught driving under the alcohol influence or not.

Moreover, there is only one mascot representing each team, but it is possible to have a team without any. Also, every security guard works on a single arena, but as it is obvious, many of them work on each one.

Finally, we want to keep track of every match and ticket sales. We want to save every match date and result (local and visitor points). Take into account any other information related to the match (when, where, who, …) and of course the MVP for each match should also be saved.

Our ticket sales staff are responsible for selling tickets for every match and we want to know when every ticket is sold and who did it. For every ticket we should save its id number, associated seat of the arena, if its VIP or not and the price. We will not save who bought tickets for GRPD compliance.

DELIVER YOUR PROPOSAL

In teams of **3 students**, develop:

1. **Conceptual Model** (E-R) design diagram for a database able to deal with all the requirements explained in the NBA scenario.
2. **Relational Model** diagram which matches the previous Conceptual Model diagram.
3. **SQL code scripting** which deploys the previously designed database.

Due Date: 15/11/2026 23:59h CET

For the delivery, bring in at AG:

1. A document called **challenge1-design.pdf** with the name of all the participants, the assumptions you took in the requirements and the exported pictures of both diagrams. Draw.io allows you to export the diagrams to images; please DO NOT deliver screenshots.
2. You must also include a *.png* file for each diagram for better reading and understanding: (**challenge1-mc.png**, **challenge1-mr.png**)
3. A SQL file with all the scripts used to implement the database (**challenge1-create.sql**)
4. SQL Code should also be deployed in the UPF server, connection parameters will be given to you after your team is formed.



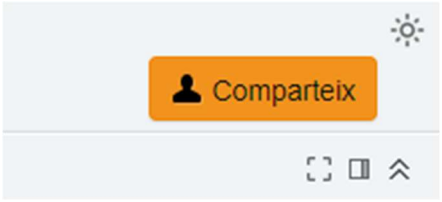
Use draw.io to develop your design diagram proposals together. Also available in the Google's Workspace Marketplace of your Google's student account.
Note: Use the *Entity-Relation* figures not the *General* ones.





Use the Entity-Relation figures (Relació de l'entitat in the image shown) except for the triangle figure in the generalisation.

You can work collaborative with your mates in draw.io using the menu on the top right:



GRADING RULES:

Conceptual Model (50 points)

Create an accurate database Conceptual Model design able to handle the provided scenario as described in Challenge 1 using the proper figures from draw.io IDE.

Correction criteria

Every design will start from the maximum mark (50 points) and there will be subtractions in the following cases:

- Poor effort or quality on the delivered document (-10)
- Missing entity or non related to the rest of the model (-2)
- An entity without a PK (-1)
- A wrong weak entity identification (-1)
- Missing generalisation or bad execution of it (-1)
- Loss of provided information, non existing attribute for an entity (-1)
- Wrong or missing relationship definition, (eg. wrong cardinality) (-2)

Relational Model (25 points)

Starting from the Conceptual Model previously designed, create the matching Relational Model according to it.

The delivered document has to contain the name of all the group participants on a front page and the designs have to be explained and defended with few lines.

Correction criteria

Every design will start from the maximum mark (25 points) and there will be subtractions in the following cases where the Relational Model doesn't match the Conceptual Model:

- Poor effort or quality on the delivered document (-10)
- Missing table (-2)
- Missing PK on a table (-1)
- Missing column from an existing attribute (-1)
- Wrong translation or missing FK (-1)
- Non matching table or column name (-1)

Creating the database (25 points)

Following the database design you proposed, you will have to decide the data types, domains of the fields (columns), primary and foreign keys for each table. Then, you have to implement the SQL commands to create the full database inside the provided database connection server.

Take into account the data types modificators such as UNSIGNED for integer columns in case of avoiding negative values, AUTO_INCREMENT, NULL or NOT NULL in case a field is accepting NULL value or not , DEFAULT value, ENUMS, VARCHAR(XX), BIT, TINYINT, MEDIUMINT, INT, etc.

The sequence of the SQL commands must respect the constraints imposed by the table's foreign keys (FK). In other words, the order of the table's creation matters. You can also decide to create all the tables with PK but without FK and later on do ALTER tables to add the FKs in the proper order.

Compose all the SQL creation commands in a file called **challenge1-create.sql** to be delivered in.

Using DROP TABLE IF EXISTS and CREATE TABLE IF NOT EXISTS statements for each table should be taken into account to avoid errors in case the table already exists before creation. After giving it a try, it is normal to have part of the database created and you may need to DROP it before trying again from scratch.

The resulting script has to have the property of being executed several times and always deliver the same result; so it is reusable and does not raise any errors.

Correction criteria

Every delivery will start from 25 points on this phase and there might be subtractions in case of:

- Source code properly deployed in the provided database server (-15)
- For every significant difference between the previous proposed database design vs the final implemented database (-10)
- Missing table (-2)
- Wrong field data type or wrong data type modifications (e.g. UNSIGNED) (-1)
- A field is missing from a table (-1)
- A PK is missing or bad definition of it (-1)
- An FK is missing or a bad definition of it (watch out the compound ones!) (-1)
- Wrong table or column name (-1)
- The sql script does not provide always the same result after several runs (-1)